

UNIVERSIDADE REGIONAL INTEGRADA DO ALTO URUGUAI E DAS MISSÕES
PRÓ-REITORIA DE ENSINO, PESQUISA, EXTENSÃO E PÓS-GRADUAÇÃO
CÂMPUS DE ERECHIM
DEPARTAMENTO DE ENGENHARIAS E CIÊNCIA DA COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ANA LUISA RIGO
GABRIEL CAMERINI
HENRIQUE OTFINOSKI
JÚLIA BARAZETTI
LUCAS BLANGER

PROJETO INTEGRADOR: DESENVOLVIMENTO E IMPLEMENTAÇÃO DE
JOGOS CLÁSSICOS NO ARDUINO UNO

ERECHIM – RS
2025

ANA LUISA RIGO
GABRIEL CAMERINI
HENRIQUE OTFINOSKI
JÚLIA BARAZETTI
LUCAS BLANGER

**PROJETO INTEGRADOR: DESENVOLVIMENTO E IMPLEMENTAÇÃO DE
JOGOS CLÁSSICOS NO ARDUINO UNO**

Trabalho elaborado e apresentado na disciplina
Projeto Integrador I, Curso de Ciências da
Computação, da Universidade Regional
Integrada do Alto Uruguai e das Missões –
Campus de Erechim.

Professor: Neilor Avelino Tonin

ERECHIM – RS
2025

RESUMO

Este projeto tem como objetivo o desenvolvimento e a implementação de jogos clássicos interativos utilizando a plataforma Arduino Uno. Foram recriados três jogos: *Snake Game*, *Dino Runner* e *Brick Breaker*, adaptados para funcionar com recursos limitados, como o display LCD 16x2 I2C e cinco botões físicos, além de um buzzer piezoelétrico para efeitos sonoros. O sistema foi programado na Arduino IDE, com linguagem C++, e todos os jogos foram projetados para permitir interação intuitiva do usuário.

Durante a construção, foi utilizada a plataforma Tinkercad para simular e validar os circuitos antes da montagem física em protoboard. O projeto proporcionou aos integrantes da equipe a oportunidade de aplicar conhecimentos em lógica de programação, eletrônica básica, estruturação de código e criação de interfaces em sistemas embarcados. Entre os desafios enfrentados estiveram o controle preciso de entrada dos botões, o limite de espaço do display e a sincronização dos elementos gráficos.

O resultado final apresenta um sistema funcional, com jogos nostálgicos operando de forma fluida no Arduino Uno, evidenciando o potencial da plataforma como ferramenta didática. A proposta se mostra promissora tanto para o ensino de fundamentos técnicos quanto como base para o desenvolvimento de jogos mais avançados com microcontroladores.

Palavras-Chave: Arduino Uno; Jogos Clássicos; Programação Embarcada; Interface Física; Display LCD.

SUMÁRIO

RESUMO	3
1 INTRODUÇÃO	4
2 OBJETIVOS	4
2.1 Objetivo Geral	4
2.2 Objetivos Específicos	4
3 REFERENCIAL TEÓRICO	5
3.1 História dos Jogos Clássicos	5
3.2 Materiais Utilizados	5
3.2.1 Arduino Uno	5
3.2.2 Display LCD 16x2 I2C	5
3.2.3 Botões e Alto-Falante	6
3.2.3 Protoboard e Cabeamento	6
3.3 Arduino IDE	6
3.4 Modelo Elaborado no Tinkercad	6
4 METODOLOGIA	7
4.1 Etapa de Planejamento	7
4.2 Testes Realizados	7
4.3 Etapa de Desenvolvimento	7
5 DESENVOLVIMENTO	8
5.1 Lógica dos Jogos	8
5.1.2 Menu Principal	8
5.1.3 Snake Game	8
5.1.4 Dino Runner	9
5.1.5 Brick Breaker	10
5.2 Mapeamento dos Botões	10
5.3 Montagem Física	11
5.4 Prints dos Jogos	12
6 CÓDIGO	14
6.1 Código do Menu Principal e Funções de Som	14
6.2 Código do Dino Runner	17
6.3 Código do Brick Breaker	19
6.4 Código do Snake Game	21
7 CONCLUSÃO	23
8 PERSPECTIVAS FUTURAS	23
9 REFERÊNCIAS	24

1 INTRODUÇÃO

O desenvolvimento de jogos clássicos em plataformas embarcadas é uma excelente forma de aliar programação, eletrônica e criatividade. Com o avanço da computação física e o interesse crescente por aplicações práticas, projetos desse tipo oferecem uma oportunidade concreta de aprendizado em lógica computacional, interação com hardware e design de sistemas simples, porém funcionais.

Foram escolhidos três jogos icônicos: **Snake Game**, **Dino Runner** e **Brick Breaker** por sua simplicidade estrutural, popularidade e facilidade de adaptação ao ambiente limitado do Arduino Uno. Utilizando apenas cinco botões físicos, um display LCD 16x2 com interface I2C e um buzzer piezoelétrico, os jogos foram implementados de forma interativa, permitindo controle intuitivo e retorno sonoro ao usuário.

Além da montagem física, o projeto incluiu simulações prévias no Tinkercad para validar circuitos e lógicas, e foi totalmente desenvolvido na Arduino IDE, empregando linguagem C++. A experiência proporcionou um contato direto com desafios comuns em sistemas embarcados, como tratamento de debounce, uso eficiente de memória e limitações de exibição. Ao final, o resultado não apenas demonstra o funcionamento dos jogos, mas também serve como base para projetos mais robustos e com maior complexidade gráfica e sonora.

2 OBJETIVOS

2.1 Objetivo Geral

Desenvolver jogos clássicos interativos no Arduino Uno, utilizando botões físicos como entrada e display LCD 16x2 I2C como saída visual.

2.2 Objetivos Específicos

- Desenvolver e implementar o jogo Dino Runner.
- Desenvolver e implementar o jogo Snake Game.
- Desenvolver e implementar o jogo Brick Breaker.
- Exibir informações dos jogos no LCD 16x2 I2C.

3 REFERENCIAL TEÓRICO

3.1 História dos Jogos Clássicos

Dino Runner: Popularizado pelo navegador Google Chrome, o Dino Runner é um jogo de corrida infinita no qual o jogador controla um dinossauro que deve desviar de obstáculos, como cactos e pássaros, enquanto percorre um deserto. O jogo foi criado como uma forma de entretenimento offline, quando não há conexão com a internet.

Snake Game (Jogo da Cobrinha): Surgiu em 1976, com o lançamento do jogo Blockade pela empresa Gremlin. Ganhou enorme popularidade nos anos 2000 com as versões presentes nos celulares Nokia, especialmente no modelo Nokia 6110. O objetivo do jogo é controlar uma cobra que cresce cada vez que come um item, evitando colisões com as paredes ou com o próprio corpo.

Brick Breaker: Inspirado no clássico jogo Breakout, lançado pela Atari em 1976, o Brick Breaker consiste em quebrar blocos posicionados na parte superior da tela utilizando uma bola, que é refletida por uma barra controlada pelo jogador. A cada bloco destruído, o jogador acumula pontos e o desafio aumenta.

3.2 Materiais Utilizados

3.2.1 Arduino Uno

O Arduino Uno é uma placa de prototipagem com microcontrolador ATmega328P. Possui 14 portas digitais, 6 entradas analógicas, conexão USB e funciona com 5V. É amplamente utilizado por iniciantes e profissionais devido à sua simplicidade e versatilidade.

3.2.2 Display LCD 16x2 I2C

Permite exibir 2 linhas com 16 caracteres. A versão I2C utiliza apenas 4 fios (VCC, GND, SDA, SCL), simplificando a conexão. A biblioteca **LiquidCrystal_I2C** facilita a integração com o Arduino.

3.2.3 Botões e Alto-Falante

Botões são conectados às entradas digitais e utilizados para capturar comandos. O alto-falante emite sons por meio da função *tone()*, dando feedback ao usuário durante o jogo.

3.2.3 Protoboard e Cabeamento

Protoboard é uma ferramenta usada para montar circuitos eletrônicos temporários, sem a necessidade de solda. Jumpers foram utilizados para interligar os componentes à placa Arduino.

3.3 Arduino IDE

A Arduino IDE é o ambiente de desenvolvimento utilizado para programar placas como o Arduino Uno. Gratuita e de fácil utilização, ela permite escrever, compilar e enviar códigos diretamente para o microcontrolador.

Seu funcionamento é baseado na linguagem C/C++, com suporte a diversas bibliotecas que facilitam o uso de sensores, displays e atuadores. Além disso, a IDE possui recursos como monitor serial, exemplos prontos e uma interface simples.

3.4 Modelo Elaborado no Tinkercad

Antes da montagem física do circuito, foi realizada a simulação do projeto na plataforma Tinkercad, ferramenta online de prototipagem eletrônica. Essa etapa permitiu validar o funcionamento da lógica de conexão entre os componentes, como o Arduino Uno, o display LCD, os botões de controle e o buzzer, além de testar o comportamento básico dos sinais de entrada e saída.

A simulação no Tinkercad facilitou a identificação de possíveis erros de ligação e contribuiu para um processo de montagem mais seguro e eficiente na protoboard.

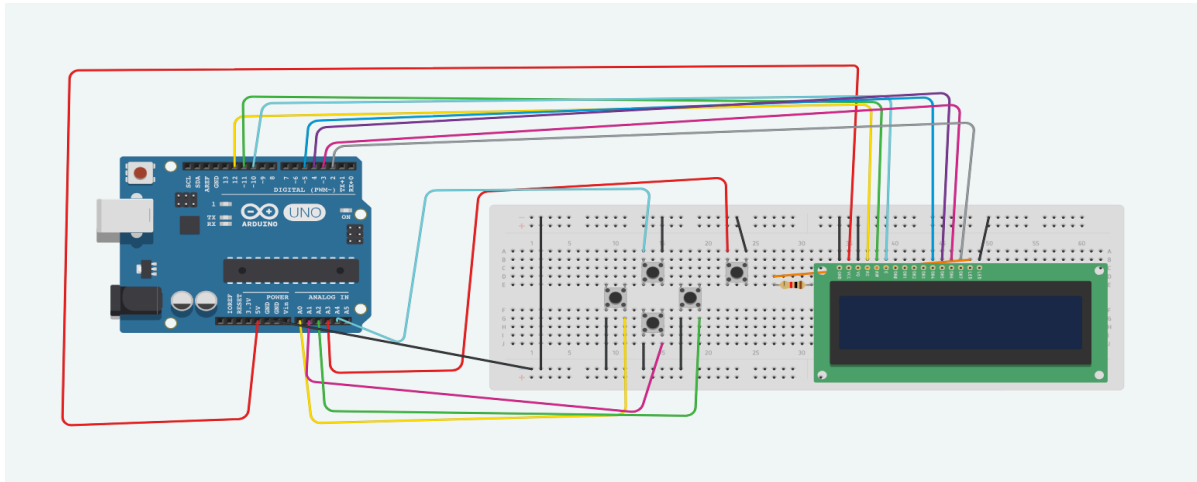


Figura 1: Imagem do projeto na plataforma Tinkercad

4 METODOLOGIA

4.1 Etapa de Planejamento

Seleção dos jogos: Dino Runner, Snake Game Brick Breaker.

Definição dos materiais: Arduino Uno, LCD 16x2 I2C, alto-falante, botões, jumpers e protoboard.

4.2 Testes Realizados

- Testes individuais de cada jogo.
- Testes com os botões.
- Teste de funcionamento do LCD.

4.3 Etapa de Desenvolvimento

- Implementação da lógica de cada jogo na IDE do Arduino, utilizando linguagem C++.
- Leitura dos botões digitais e tratamento dos comandos.
- Exibição das informações no display LCD.
- Implementação de sons com o alto-falante.
- Montagem física dos circuitos.

5 DESENVOLVIMENTO

5.1 Lógica dos Jogos

5.1.2 Menu Principal

O menu principal é o ponto de entrada do sistema, onde o usuário pode selecionar qual jogo deseja jogar. Ele utiliza o LCD 16x2 I2C para exibir as opções de jogos e os botões físicos (Cima, Baixo e OK) para navegar entre as opções.

A navegação entre os itens do menu é feita com os botões "Cima" e "Baixo". O Arduino usa operadores modulares para garantir que a seleção "cicle" entre as opções (ex: depois da última opção, volta para a primeira).

Toda vez que o jogador muda de opção ou confirma, o sistema gera um som curto com o buzzer, através da função *selectSound()*. No LCD é exibido, linha 1, "Selecione o jogo:" e na linha 2 o nome do jogo atualmente selecionado (exemplo: "Snake", "Dino", "Brick").

Portanto, a estrutura principal utiliza as seguintes funções: *showMenu()*, *showBemvindo()*, *selectSound()* e com uma condicional de escolha de jogo dentro da função *loop()*.

5.1.3 Snake Game

O Snake Game implementa a tradicional lógica de "Cobrinha", adaptada ao espaço de 2 linhas por 16 colunas do LCD.

- Estrutura de Dados:
 - O jogo usa dois arrays:
 - *snakeX[]*: Posições X da cabeça e corpo da cobra
 - *snakeY[]*: Posições Y (linha) da cabeça e corpo da cobra
 - Tamanho máximo definido pela constante *LARGURA_MAX*.
- Movimentação:
 - O jogador usa os botões Cima, Baixo, Esquerda, Direita para alterar a direção da cobra.
 - O código impede a inversão direta (ex: de cima para baixo direto).
- Crescimento da Cobra:
 - Cada vez que a cobra "come" a comida, o tamanho (largura) aumenta.
 - A comida é reposicionada aleatoriamente usando *random()*.
- Detecção de Colisão:

- Colisão com parede: Se sair da área 16x2 é *Game Over*.
- Colisão com o próprio corpo: Também retorna *Game Over*.
- Visualização:
 - A cobra é representada com o caractere "*", e a comida com um caractere personalizado (bloco).
- Feedback Sonoro:
 - Ao comer: Som curto (função *beep()*).
 - Ao perder: Som de *Game Over* (*gameOverSound()*).

5.1.4 Dino Runner

O Dino Runner é um jogo de desvio de obstáculos, com um dinossauro que pode pular.

- Personagem:
 - O dinossauro é exibido com um caractere personalizado no LCD, nas linhas 0 (pulo) ou 1 (chão).
- Obstáculo:
 - Um obstáculo também representado com caractere personalizado, que se move da direita para a esquerda (posição *obstaculosPos* de 15 a 0).
- Controle:
 - O jogador usa o botão OK para fazer o dinossauro "pular".
- Lógica de Pulo:
 - O código usa uma variável pulando e um contador de frames (*jumpFrame*) para controlar o tempo que o dinossauro permanece no ar.
- Detecção de Colisão:
 - Se o obstáculo alcançar a posição 1 na linha de chão enquanto o dinossauro não estiver pulando, é *Game Over*.
- Pontuação:
 - O número de obstáculos ultrapassados é exibido no canto superior direito do LCD.
- Sons:
 - Ao pular: *pularSom()*.
 - Ao perder: *gameOverSound()*.

5.1.5 Brick Breaker

O Brick Breaker simula o jogo de quebrar blocos com uma bola que rebate.

- Estrutura de Blocos:
 - Um array *bricks[]*, com 16 posições, representa os blocos na linha de cima do LCD.
- Paddle (Barra do Jogador):
 - O jogador controla a barra usando os botões Esquerda e Direita, movimentando a posição *pPos* da barra de 3 colunas.
- Bola:
 - A bola tem coordenadas *bolaX* (coluna) e *bolaY* (linha), e direções *dirX* e *dirY*.
 - Ela se move a cada ciclo, rebatendo nas paredes, na barra e nos blocos.
- Colisão com os blocos:
 - Ao colidir com um bloco, o bloco some (posição no array vira false), a bola rebate e o jogador ouve um *beep()*.
- Derrota:
 - Se a bola passar da barra e sair da tela (linha maior que 1), dá *Game Over*.
- Vitória:
 - Se todos os blocos forem destruídos: Vitória, com som de celebração.
- Controle de Velocidade:
 - O movimento da bola é controlado por *delay()* no loop.

5.2 Mapeamento dos Botões

O projeto utilizou cinco botões físicos para a interação do usuário com os jogos. Cada botão foi mapeado para uma função específica, de acordo com a lógica de controle de cada jogo. A tabela a seguir apresenta a correspondência entre os botões físicos e suas respectivas funcionalidades no código:

Botão	Função no Jogo	Pino Digital do Arduino
-------	----------------	-------------------------

Botão 1	Cima	4
Botão 2	Baixo	5
Botão 3	Esquerdo	3
Botão 4	Direita	6
Botão 5	Confirmação/Pular	2

Quadro 1: Mapeamento dos Botões

O sistema de leitura dos botões foi implementado utilizando o modo *INPUT_PULLUP*, aproveitando os resistores internos do Arduino para simplificar o circuito.

5.3 Montagem Física

A montagem física do projeto foi realizada utilizando uma protoboard de tamanho grande, o que facilitou a disposição e a organização dos componentes eletrônicos.

Distribuição dos Componentes:

- Display LCD 16x2 com interface I2C:
 - Posicionado na parte superior da protoboard, facilitando a visualização por parte do usuário. O display foi conectado ao Arduino Uno através de apenas quatro fios: VCC, GND, SDA e SCL, simplificando o cabeamento.
- Botões de Controle:
 - Foram utilizados cinco botões *push-button*, distribuídos nas colunas centrais da protoboard, de forma espaçada para facilitar o manuseio durante a interação com os jogos. Cada botão foi conectado a um pino digital do Arduino (com configuração em *INPUT_PULLUP*), além de serem ligados ao GND para garantir o funcionamento correto.
- Buzzer Piezoelétrico:

- Instalado na lateral direita da protoboard, próximo ao display. O buzzer foi conectado ao pino digital 8 do Arduino, sendo responsável pela emissão dos sons de interação e feedback durante os jogos.
- Alimentação:
 - Todo o circuito foi alimentado através da própria conexão USB do Arduino Uno, visível no canto esquerdo da montagem.
- Fiação:
 - Foram utilizados jumpers macho-macho para realizar todas as conexões entre os botões, o LCD, o buzzer e os pinos do Arduino. As cores dos fios foram variadas, ajudando na identificação visual dos circuitos durante a montagem e os testes.

A montagem priorizou a organização dos cabos e a disposição lógica dos componentes, visando facilitar tanto os testes quanto eventuais manutenções ou modificações futuras.

5.4 Prints dos Jogos

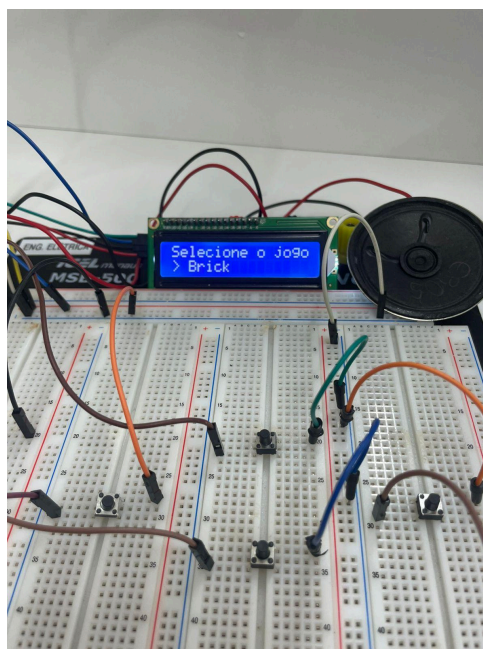


Figura 2: Imagem do menu do fliperama em funcionamento

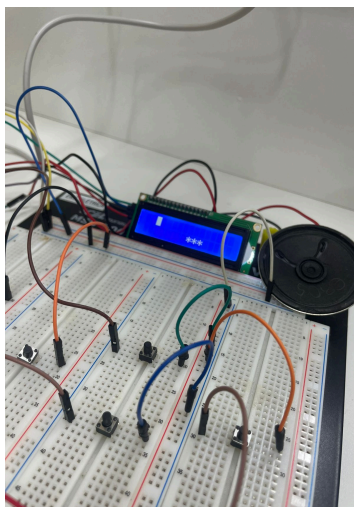


Figura 3: Imagem do jogo Snake game em funcionamento

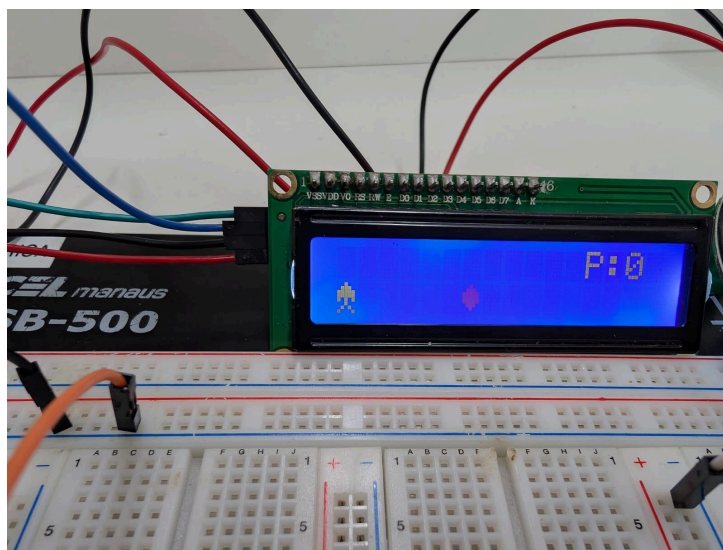


Figura 4: Imagem do jogo Dino Runner em funcionamento

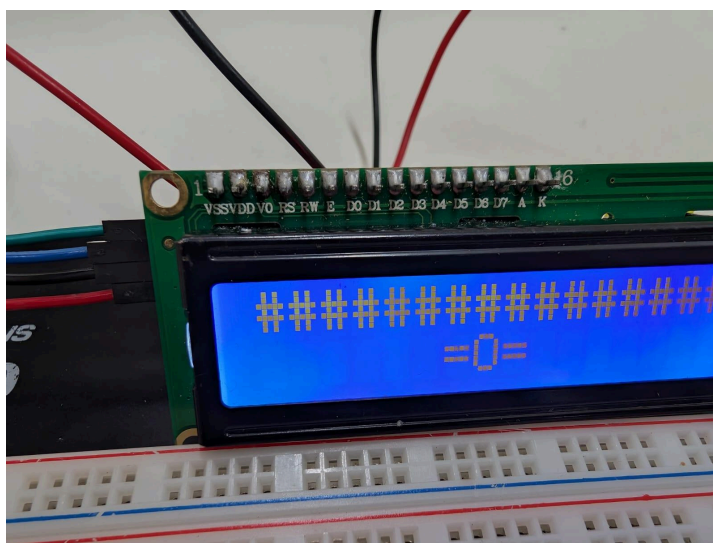


Figura 5: Imagem do jogo Brick Breaker em funcionamento

6 CÓDIGO

Nesta seção, estão apresentados os códigos completos desenvolvidos para a implementação dos jogos clássicos no Arduino Uno.

6.1 Código do Menu Principal e Funções de Som

```
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const int btnCima = 4;
const int btnBaixo = 5;
const int btnDireita = 6;
const int btnEsquerda = 3;
const int btnOk = 2;
const int buzzerPin = 8;

byte block[] = {
  B11111,
  B11111,
  B11111,
  B11111,
  B11111,
  B11111,
  B11111,
}
```

```

    B11111,
    B11111
};

int pontos;

void setup();
void loop();

void beep(int freq, int duration);
void gameOverSound();
void vitoriaSom();
void pularSom();
void selectSound();

void showBemvindo();
void showMenu();

void startDino();
void startBrickBreaker();

int opcao = 0;
const int numOpcao = 3;
String opcoes[] = {"Snake", "Dino", "Brick"};
void setup() {
    lcd.init();
    lcd.backlight();

    pinMode(btnCima, INPUT_PULLUP);
    pinMode(btnBaixo, INPUT_PULLUP);
    pinMode(btnOk, INPUT_PULLUP);
    pinMode(btnEsquerda, INPUT_PULLUP);
    pinMode(btnDireita, INPUT_PULLUP);
    pinMode(buzzerPin, OUTPUT);

    lcd.createChar(0, block);

    showBemvindo();
    showMenu();
}
void loop() {
    if (digitalRead(btnCima) == LOW) {
        opcao = (opcao - 1 + numOpcao) % numOpcao;
        selectSound();
        showMenu();
        delay(200);
    }
    if (digitalRead(btnBaixo) == LOW) {

```



```

    opcao = (opcao + 1) % numOpcao;
    selectSound();
    showMenu();
    delay(200);
}
if (digitalRead(btnOk) == LOW) {
    selectSound();
    switch (opcao) {
        case 0: startSnake(); break;
        case 1: startDino(); break;
        case 2: startBrickBreaker(); break;
    }
    showMenu();
}
}

void beep(int freq, int duration) {
    tone(buzzerPin, freq, duration);
    delay(duration);
    noTone(buzzerPin);
}

void gameOverSound() {
    beep(200, 200);
    beep(150, 300);
    beep(100, 400);
}

void vitoriaSom() {
    beep(500, 200);
    beep(700, 200);
    beep(900, 300);
}

void pularSom() {
    beep(600, 100);
}

void selectSound() {
    beep(1000, 80);
}

void showBemvindo() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(" Bem-vindo!");
    lcd.setCursor(0, 1);
    lcd.print(" Carregando");
    for (int i = 0; i < 6; i++) {
        lcd.print(".");
        delay(440);
    }
    delay(500);
    lcd.clear();
}

```

```

}
void showMenu() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Selecione o jogo:");
    lcd.setCursor(0, 1);
    lcd.print("> " + opcoes[opcao]);
}

```

6.2 Código do Dino Runner

```

bool pulando = false;
int jumpFrame = 0;
int obstaculosPos = 15;
int pontosDino = 0;
bool gameOverDino = false;

// Define caractere dinossauro (índice 0)
byte dinoChar[8] = {
    B00100,
    B01110,
    B11111,
    B11111,
    B10101,
    B00100,
    B01010,
    B10001
};

// Define caractere obstáculo (índice 1)
byte obstaculoChar[8] = {
    B00100,
    B01110,
    B11111,
    B11111,
    B11111,
    B01110,
    B00100,
    B00000
};

void startDino() {
    lcd.clear();
    lcd.createChar(0, dinoChar);
    lcd.createChar(1, obstaculoChar);

    lcd.setCursor(0, 0);
    lcd.print("Dino Runner");
    delay(1000);
}

```

```

lcd.clear();

pulando = false;
jumpFrame = 0;
obstaculosPos = 15;
pontosDino = 0;
gameOverDino = false;

while (!gameOverDino) {
    lcd.clear();

    if (pulando) {
        lcd.setCursor(0, 0);
        lcd.write(byte(0)); // dinossauro pulando (linha 0)
    } else {
        lcd.setCursor(0, 1);
        lcd.write(byte(0)); // dinossauro no chão (linha 1)
    }
    if (obstaculosPos >= 0) {
        lcd.setCursor(obstaculosPos, 1);
        lcd.write(byte(1)); // obstáculo na linha 1
    }
    if (!pulando && obstaculosPos == 1) {
        gameOverSound();
        lcd.setCursor(5, 0);
        lcd.print("GAME OVER");
        gameOverDino = true;
        break;
    }
    if (digitalRead(btnOk) == LOW) {
        if (!pulando) {
            pulando = true;
            pularSom();
        }
    }
    if (pulando) {
        jumpFrame++;
        if (jumpFrame > 2) {
            pulando = false;
            jumpFrame = 0;
        }
    }
    obstaculosPos--;
    if (obstaculosPos < 0) {
        obstaculosPos = 15;
        pontosDino++;
    }
    lcd.setCursor(12, 0);
    lcd.print("P:");

```

```

    lcd.print(pontosDino);
    delay(200);
}
delay(1500);
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("P: ");
lcd.print(pontosDino);
delay(2000);
}

```

6.3 Código do Brick Breaker

```

void startBrickBreaker() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Brick Breaker");
    delay(1000);

    bool bricks[16];
    for (int i = 0; i < 16; i++) bricks[i] = true;

    int bolaX = 7;
    int bolaY = 1;
    int dirX = 1;
    int dirY = -1;
    int pPos = 6;

    while (true) {
        lcd.clear();

        for (int i = 0; i < 16; i++) {
            if (bricks[i]) {
                lcd.setCursor(i, 0);
                lcd.print("#");
            }
        }

        lcd.setCursor(pPos, 1);
        for (int i = 0; i < 3; i++) {
            if (pPos + i < 16)
                lcd.print("=");
        }

        if (bolaX >= 0 && bolaX < 16 && bolaY >= 0 && bolaY < 2) {
            lcd.setCursor(bolaX, bolaY);
            lcd.print("O");
        }
    }
}

```

```

delay(600);

bolaX += dirX;
bolaY += dirY;

if (bolaX <= 0 || bolaX >= 15) {
    dirX *= -1;
    beep(700, 50);
}
if (bolaY <= 0) {
    dirY = 1;
    beep(700, 50);
}

if (bolaY == 1 && bolaX >= pPos && bolaX <= pPos + 2) {
    dirY = -1;
    beep(700, 80);
}

if (bolaY == 0 && bolaX >= 0 && bolaX < 16) {
    if (bricks[bolaX]) {
        bricks[bolaX] = false;
        dirY = 1;
        beep(300, 50);
    }
}

if (digitalRead(btnEsquerda) == LOW && pPos > 0) {
    pPos--;
    delay(500);
}
if (digitalRead(btnDireita) == LOW && pPos < 13) {
    pPos++;
    delay(500);
}

if (bolaY > 1) {
    gameOverSound();
    lcd.clear();
    lcd.setCursor(3, 0);
    lcd.print("GAME OVER");
    delay(2000);
    return;
}

bool venceu = true;
for (int i = 0; i < 16; i++) {
    if (bricks[i]) {

```

```

        venceu = false;
        break;
    }
}

if (venceu) {
    vitoriaSom();
    lcd.clear();
    lcd.setCursor(4, 0);
    lcd.print("VENCEU!");
    delay(2000);
    return;
}
}
}

```

6.4 Código do Snake Game

```

void startSnake() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("SNAKE GAME");
    delay(1000);
    lcd.clear();

    const int LARGURA_MAX = 32;
    int snakeX[LARGURA_MAX];
    int snakeY[LARGURA_MAX];
    int largura = 3;

    int dirX = 1;
    int dirY = 0;

    int comidaX = random(0, 16);
    int comidaY = random(0, 2);

    for (int i = 0; i < largura; i++) {
        snakeX[i] = 5 - i;
        snakeY[i] = 1;
    }

    bool gameOver = false;

    while (!gameOver) {
        if (digitalRead(btnCima) == LOW && dirY != 1) { dirX = 0; dirY = -1; }
        if (digitalRead(btnBaixo) == LOW && dirY != -1) { dirX = 0; dirY = 1; }
        if (digitalRead(btnEsquerda) == LOW && dirX != 1) { dirX = -1; dirY = 0; }
        if (digitalRead(btnDireita) == LOW && dirX != -1) { dirX = 1; dirY = 0; }
    }
}

```

```

for (int i = largura - 1; i > 0; i--) {
    snakeX[i] = snakeX[i - 1];
    snakeY[i] = snakeY[i - 1];
}
snakeX[0] += dirX;
snakeY[0] += dirY;

if (snakeX[0] < 0 || snakeX[0] >= 16 || snakeY[0] < 0 || snakeY[0] > 1) {
    gameOver = true;
    break;
}
for (int i = 1; i < largura; i++) {
    if (snakeX[0] == snakeX[i] && snakeY[0] == snakeY[i]) {
        gameOver = true;
        break;
    }
}
if (snakeX[0] == comidaX && snakeY[0] == comidaY) {
    if (largura < LARGURA_MAX) largura++;
    comidaX = random(0, 16);
    comidaY = random(0, 2);
    beep(1000, 100);
}

lcd.clear();
lcd.setCursor( comidaX, comidaY);
lcd.write(byte(0));
for (int i = 0; i < largura; i++) {
    lcd.setCursor(snakeX[i], snakeY[i]);
    lcd.print("");
}

delay(500);
}

gameOverSound();
lcd.clear();
lcd.setCursor(2, 0);
lcd.print("GAME OVER");
lcd.setCursor(2, 1);
lcd.print("Pontos: ");
lcd.print(largura);
delay(2000);
}

```

7 CONCLUSÃO

O desenvolvimento e a implementação de jogos clássicos no Arduino Uno proporcionaram uma experiência prática e multidisciplinar, unindo conceitos de programação, eletrônica e design de interface. Ao recriar os jogos consagrados utilizando recursos limitados como o display LCD 16x2, cinco botões físicos e um buzzer a equipe conseguiu superar desafios técnicos e propor soluções criativas dentro do contexto de sistemas embarcados.

Durante o processo, foi possível explorar de forma aprofundada a lógica de jogos, o uso eficiente da memória do microcontrolador, o mapeamento e tratamento de entradas digitais, além da importância de uma boa prototipação, realizada inicialmente na plataforma Tinkercad. A programação na Arduino IDE também permitiu aplicar conhecimentos de C++, modularização de código e controle de fluxo para jogos em tempo real.

Apesar das limitações de hardware, o projeto demonstrou que é possível criar jogos funcionais e interativos com poucos componentes, sendo uma excelente ferramenta para fins educacionais e introdutórios à computação física. As dificuldades encontradas como o debounce dos botões, a baixa resolução do display e o controle da sincronização serviram como aprendizado técnico valioso para os participantes.

Conclui-se, portanto, que o projeto alcançou com sucesso seus objetivos e abriu espaço para futuras melhorias e expansões. Ele também reforça o valor de plataformas como o Arduino no ensino de computação, eletrônica e lógica de forma acessível, prática e divertida.

8 PERSPECTIVAS FUTURAS

- **Portabilidade com bateria recarregável:** adaptar o sistema para funcionar de forma autônoma, utilizando baterias Li-Ion ou Li-Po e módulos de carregamento.
- **Gravação de pontuação:** implementar memória EEPROM ou cartão SD para salvar recordes ou histórico de partidas.

- **Interface com smartphone via Bluetooth:** adicionar um módulo Bluetooth (como o HC-05) para controlar os jogos por um aplicativo mobile ou enviar dados ao celular.
- **Criação de uma “carcaça” estilo fliperama:** projetar e imprimir em 3D uma estrutura física personalizada para abrigar o sistema e tornar a experiência mais imersiva.
- **Adicionar novos efeitos sonoros:** ampliar a variedade de sons para ações como vitória, colisão ou mudança de fase, enriquecendo a experiência do usuário.
- **Migrar para um display gráfico (OLED ou TFT):** substituir o LCD 16x2 por uma tela gráfica, permitindo uma representação visual mais rica e detalhada dos jogos.
- **Otimizar o uso de memória:** revisar os códigos para melhorar o desempenho e evitar estouros de memória RAM e EEPROM do Arduino.

9 REFERÊNCIAS

SPARKFUN. “L298_H_Bridge”. Disponível em: www.sparkfun.com. Acesso em: 05 out. 2022.

ROBOCORE. “BlackBoard”. Disponível em: www.robocore.net. Acesso em: 10 out. 2022.

SOLDAFRIA. “O que é o Arduino”. Disponível em: www.soldafria.com.br. Acesso em: 18 out. 2022.

ARDUINO.CC. “Arduino Uno”. Disponível em: <https://www.arduino.cc>. Acesso em: 10 mai. 2025.